



SOFTWARE REQUIREMENTS SPECIFICATION

Movie Search App



A Web-Based Movie Discovery Application using HTML5, CSS3 & JavaScript (ES6)



Document Version	1.0
Standard Followed	IEEE 830-1998 guidelines
Technology Stack	HTML5 · CSS3 · JavaScript (ES6)
Prepared For	GLA University, Greater Noida
Prepared By	Divyanshi (NC)
Date	18-08-2026

Table of Contents

1.	Introduction
2.	Overall Description
3.	System Features & Functional Requirements
4.	External Interface Requirements
5.	Non-Functional Requirements
6.	System Architecture & Design
7.	Data Requirements & API Planning
8.	Use Case Specifications
9.	UI / UX Design Requirements
10.	Software Testing Plan
11.	Project Constraints, Assumptions & Future Scope
12.	Appendix

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the **Movie Search App**, a web-based application that allows users to search for movies and view detailed information about them in real time. The purpose of this document is to provide a clear, complete, and testable description of the system for use by students (developers), faculty (evaluators), and any future contributors to the project. It defines what the system will do, how it should behave, and the constraints under which it must operate.

1.2 Scope

The Movie Search App is a single-page, client-side web application built using **HTML5, CSS3, and vanilla JavaScript (ES6+)**. Users can enter a movie title into a search bar, and the application will query a third-party movie API (e.g., OMDb API or TMDb API) using the JavaScript `Fetch API`, retrieve matching results, and dynamically render them on the page using DOM manipulation.

The application displays, for each movie: **poster image, title, release year, genre, IMDb-style rating, and plot summary**. The interface is responsive and works across desktop, tablet, and mobile screen sizes.

In Scope: Movie search by title, dynamic result rendering, movie detail view, responsive UI, loading/error states, basic client-side history of recent searches.

Out of Scope (Phase 1): User authentication, payment/booking systems, server-side database, user reviews, watch-list synchronization across devices. These are listed as future enhancements in Section 11.

1.3 Intended Audience

Audience	Purpose of Reading this Document
Student Developer(s)	Use this SRS as the primary reference for implementation, sprint planning, and module division.
Faculty / Project Guide	Evaluate scope, feasibility, technical depth, and alignment with software engineering principles.
Project Evaluators / Examiners	Assess requirement completeness, design correctness, and testing coverage during viva/demo.
Future Contributors	Understand system behavior before extending the application in later phases.

1.4 Definitions, Acronyms and Abbreviations

Term	Description
API	Application Programming Interface — used to fetch movie data from an external provider.
DOM	Document Object Model — the in-browser representation of the HTML page manipulated via JavaScript.
JSON	JavaScript Object Notation — the data format returned by the movie API.
SPA	Single Page Application — the app runs within a single HTML page without full reloads.
UI / UX	User Interface / User Experience.
SRS	Software Requirements Specification (this document).
OMDb / TMDb	Open Movie Database / The Movie Database — public movie data APIs.
Debounce	A technique that delays function execution until the user stops typing, reducing API calls.

1.5 References

- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications.
- OMDb API Documentation — <https://www.omdbapi.com/>
- The Movie Database (TMDb) API Documentation — <https://developer.themoviedb.org/>
- MDN Web Docs — Fetch API, DOM Manipulation, Responsive Web Design (developer.mozilla.org).

1.6 Document Overview

Section 2 gives an overall description of the product and its users. Section 3 defines detailed functional requirements grouped by feature. Section 4 covers external interfaces including the API contract. Section 5 defines non-functional requirements. Section 6 presents the system architecture. Section 7 covers data and API planning. Section 8 provides use case specifications. Section 9 defines UI/UX requirements. Section 10 outlines the testing plan, and Section 11 lists constraints, assumptions, and future scope.

2. Overall Description

2.1 Product Perspective

The Movie Search App is a standalone, self-contained frontend web application. It is not part of a larger system, but it depends on an external, publicly available movie data API as its data source. All business logic relevant to search, filtering, and rendering runs entirely on the client side (in the browser); there is no custom backend server or database in Phase 1.

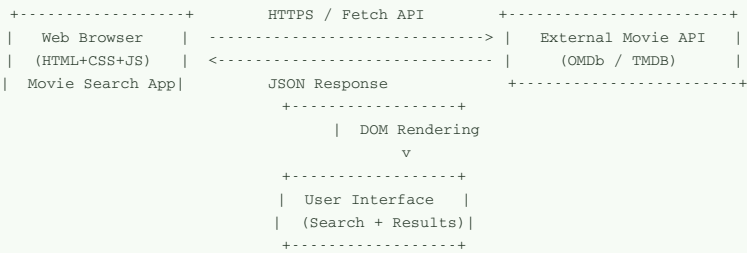


Fig 2.1 — High-level product context diagram.

2.2 Product Functions (Summary)

- Search movies by title through a text input field.
- Fetch matching movie data from an external API in real time.
- Display a responsive grid/list of matching movie results (poster + title + year).
- Show detailed movie information (rating, genre, plot, cast, runtime) on selection.
- Handle "no results found," loading, and network/API error states gracefully.
- Maintain a short client-side history of recent searches (optional enhancement).
- Adapt layout responsively across mobile, tablet, and desktop screens.

2.3 User Classes and Characteristics

User Class	Description	Technical Expertise
General Visitor / Movie Enthusiast	Primary end user who searches for movies to view information before watching.	Low — must be usable without instructions.
Student Developer	Builds, maintains, and extends the application.	Intermediate — familiar with HTML/CSS/JS.
Faculty Evaluator	Reviews the application for functional correctness and design quality.	High — technical reviewer.

2.4 Operating Environment

- **Client Side:** Any modern web browser — Google Chrome, Mozilla Firefox, Microsoft Edge, Safari (latest two versions).
- **Devices:** Desktops, laptops, tablets, and smartphones (responsive design, minimum supported width: 320px).
- **Network:** Active internet connection required for API calls.
- **Hosting (optional):** Static hosting platforms such as GitHub Pages, Netlify, or Vercel for deployment/demo.

2.5 Design and Implementation Constraints

- Must be implemented using only HTML5, CSS3, and vanilla JavaScript (no heavy frameworks required, consistent with the academic scope).
- Must use the Fetch API (or Axios, optionally) for all HTTP requests — no page reloads for search actions.
- Dependent on the rate limits and data accuracy of the chosen third-party movie API.
- API key must be handled client-side for this academic version; not intended for production-scale public deployment without a backend proxy.

2.6 Assumptions and Dependencies

- The external movie API remains available, and its endpoint/response structure does not change during development.
- A free-tier API key is sufficient for development, testing, and classroom demonstration purposes.
- Users have a stable internet connection while using the application.

3. System Features & Functional Requirements

Each feature below includes a description, priority, and detailed functional requirements (FR) with unique IDs for traceability during development and testing.

3.1 Feature: Movie Search

Description: Allows the user to type a movie title and retrieve matching results from the movie API.

ID	Requirement	Priority
FR-1.1	The system shall provide a text input field for the user to enter a movie title.	High
FR-1.2	The system shall provide a "Search" button and shall also trigger search on pressing the Enter key.	High
FR-1.3	The system shall send an asynchronous request to the movie API using the Fetch API when a search is triggered.	High
FR-1.4	The system shall not trigger a search request when the input field is empty; it shall display a prompt message instead.	Medium
FR-1.5	The system shall debounce live-search input (if implemented) by at least 400ms to limit redundant API calls.	Low

3.2 Feature: Search Results Display

ID	Requirement	Priority
FR-2.1	The system shall display all matching results in a responsive grid of movie cards.	High
FR-2.2	Each movie card shall display, at minimum: poster thumbnail, title, and release year.	High
FR-2.3	If a poster image is unavailable from the API, the system shall display a default placeholder image.	Medium
FR-2.4	The system shall display a "No movies found" message when the API returns zero results.	High
FR-2.5	The system shall display a loading indicator/spinner while the API request is in progress.	High

3.3 Feature: Movie Detail View

ID	Requirement	Priority
FR-3.1	The system shall allow the user to click/tap a movie card to view full details.	High
FR-3.2	The detail view shall display: full title, poster, release year, genre, IMDb rating, runtime, plot summary, and cast (if available from API).	High
FR-3.3	The system shall provide a clear way to close/return from the detail view to the results list (button or modal close action).	Medium

3.4 Feature: Error & Edge Case Handling

ID	Requirement	Priority
FR-4.1	The system shall display a user-friendly error message if the API request fails (network error, timeout, invalid API key).	High
FR-4.2	The system shall handle API rate-limit responses by informing the user to retry later.	Low
FR-4.3	The system shall sanitize/trim user input before sending it to the API (e.g., remove leading/trailing spaces).	Medium

3.5 Feature: Recent Search History (Optional Enhancement)

ID	Requirement	Priority
FR-5.1	The system shall store the last 5 searched movie titles in the browser's Local Storage.	Low
FR-5.2	The system shall allow the user to click a recent search term to re-run that search.	Low
FR-5.3	The system shall provide an option to clear search history.	Low

4. External Interface Requirements

4.1 User Interfaces

- A single-page interface with: header/navbar with app title, a prominent search bar, a results grid area, and a detail modal/panel.
- Consistent color theme, typography, and spacing across all screens (see Section 9 for UI/UX specification).
- All interactive elements (buttons, cards) shall provide visual feedback on hover/focus/tap.

4.2 Hardware Interfaces

No special hardware interfaces are required. The application runs entirely within a standard web browser on any internet-connected device (desktop, laptop, tablet, or smartphone).

4.3 Software Interfaces

Interface	Description
Movie Data API (OMDb / TMDb)	RESTful HTTP API returning JSON movie data, consumed via the JavaScript Fetch API. Requires an API key passed as a query parameter or header.
Browser Local Storage API	Used to persist recent search history and (optionally) a favorites/watch-list on the client device.

4.3.1 Sample API Request (illustrative — OMDb-style)

```
GET https://www.omdbapi.com/?apikey={API_KEY}&s={movie_title}

Response (200 OK):
{
  "Search": [
    {
      "Title": "Inception",
      "Year": "2010",
      "imdbID": "tt1375666",
      "Poster": "https://.../poster.jpg"
    }
  ],
  "totalResults": "1",
  "Response": "True"
}
```

4.3.2 Sample Detail Request

```
GET https://www.omdbapi.com/?apikey={API_KEY}&i={imdbID}&plot=full

Response (200 OK):
{
  "Title": "Inception",
  "Year": "2010",
  "Genre": "Action, Adventure, Sci-Fi",
  "imdbRating": "8.8",
  "Runtime": "148 min",
  "Plot": "A thief who steals corporate secrets...",
  "Poster": "https://.../poster.jpg",
  "Response": "True"
}
```

4.4 Communication Interfaces

All communication with the external API occurs over **HTTPS** using asynchronous `GET` requests. Data is exchanged exclusively in **JSON** format.

5. Non-Functional Requirements

5.1 Performance Requirements

ID	Requirement
NFR-1	Search results shall be displayed within 2 seconds under normal network conditions (excluding third-party API latency).
NFR-2	The UI shall remain responsive (non-blocking) while API calls are in progress, using asynchronous JavaScript.

5.2 Usability

ID	Requirement
NFR-3	The interface shall be intuitive enough for a first-time user to perform a search without instructions.
NFR-4	All text shall maintain a minimum contrast ratio of 4.5:1 for readability (WCAG AA guideline).

5.3 Reliability & Availability

ID	Requirement
NFR-5	The application shall not crash or freeze when the API returns an error or malformed response; it shall degrade gracefully.
NFR-6	The application shall be available for use whenever the hosting platform and the external API are online (no custom uptime guarantee required for an academic project).

5.4 Portability & Compatibility

ID	Requirement
NFR-7	The application shall function correctly on the latest two versions of Chrome, Firefox, Edge, and Safari.
NFR-8	The layout shall be fully responsive for screen widths from 320px (mobile) to 1920px (desktop) using CSS Flexbox/Grid and media queries.

5.5 Maintainability

ID	Requirement
NFR-9	Code shall be organized into separate, well-commented files (HTML structure, CSS styling, JS logic) to support future maintenance.
NFR-10	JavaScript functions shall follow single-responsibility principles (e.g., separate functions for fetching data, rendering cards, handling errors).

5.6 Security

ID	Requirement
NFR-11	The application shall sanitize user input to prevent injection into DOM rendering (avoid unsafe use of <code>innerHTML</code> with unescaped data).
NFR-12	The API key shall not be committed to public repositories in plaintext; it shall be referenced via a configuration file excluded from version control where possible.

6. System Architecture & Design

6.1 Architectural Overview

The Movie Search App follows a simple **client-side, event-driven architecture**. There is no backend server; all logic executes in the browser. The architecture can be described using three logical layers:

- **Presentation Layer (HTML/CSS):** Defines the structure and visual styling of the search bar, result cards, and detail view.
- **Logic Layer (JavaScript):** Handles event listeners, input validation, API calls (Fetch), response parsing, and DOM updates.
- **Data Layer (External API):** The third-party movie API acting as the single source of truth for movie data.

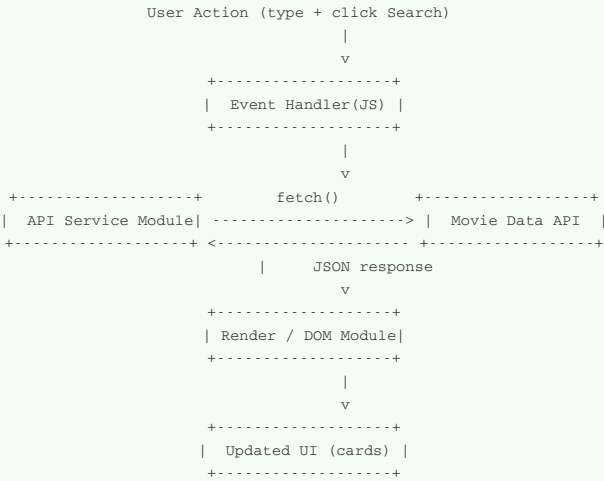


Fig 6.1 — Application logic flow diagram.

6.2 Module Breakdown

Module	File	Responsibility
Structure	index.html	Defines page structure: header, search bar, results container, modal/detail section.
Styling	style.css	Defines theme, layout (Flexbox/Grid), responsiveness, and animations.
API Service	api.js	Contains functions to build request URLs and call the Fetch API (search & detail endpoints).
Render / DOM Logic	app.js	Handles event listeners, calls API service, processes JSON, and updates the DOM dynamically.
Utilities	utils.js	Helper functions — debounce, input sanitization, local storage handling, placeholder image logic.

6.3 Recommended Folder Structure

```
movie-search-app/
├── index.html
├── /css
│   └── style.css
├── /js
│   ├── api.js
│   ├── app.js
│   └── utils.js
├── /assets
│   └── placeholder-poster.png
└── README.md
```

6.4 Technology Stack

Layer	Technology
Markup	HTML5
Styling	CSS3 (Flexbox, Grid, Media Queries)
Logic	JavaScript (ES6+), Fetch API
Data Source	OMDb API / TMDB API (public REST API, JSON)
Version Control	Git & GitHub
Deployment (optional)	GitHub Pages / Netlify / Vercel

7. Data Requirements & API Planning

7.1 Data Entities (Client-Side Model)

Although Phase 1 has no database, the application works with a structured in-memory "Movie" object mapped from the API response. This model also serves as the planning basis for a future backend database (see Section 11.3).

Field	Type	Description
id	String	Unique movie identifier from the API (e.g., imdbID).
title	String	Movie title.
year	String	Release year.
genre	String	Comma-separated list of genres.
rating	Number	IMDb-style rating (0–10 scale).
runtime	String	Movie duration (e.g., "148 min").
plot	String	Short or full plot summary.
poster	String (URL)	Link to poster image; falls back to placeholder if unavailable.
cast	String	Comma-separated list of main actors (if provided by API).

7.2 Future Database Schema (Planning Only — Phase 2)

If a future phase introduces user accounts and a personal watch-list/favorites feature, a lightweight backend (e.g., Node.js + MongoDB/Firebase) could store the following collections:

Collection	Fields
Users	userId, name, email, passwordHash, createdAt
Favorites	favoriteId, userId, movieId, title, poster, addedAt
SearchLogs (analytics, optional)	logId, userId (nullable), searchTerm, timestamp

7.3 API Selection Criteria

- Free tier availability suitable for academic/non-commercial use.
- Well-documented REST endpoints returning JSON.
- Provides poster images, ratings, genre, and plot data in a single response.
- Reasonable rate limits (sufficient for classroom demo traffic).

8. Use Case Specifications

8.1 Use Case Diagram (Textual)

```
+-----+
|           Movie Search App           |
+-----+
(User) -- (Search Movie)
|           |
|           +-- (View Results)         |
|           |
|           +-- (View Movie Details) |
|           |
|           +-- (View Recent Searches)|
|           |
|           +-- (Handle No Results / Errors)|
+-----+
```

8.2 UC-1: Search for a Movie

Use Case ID	UC-1
Actor	End User
Precondition	User has the application open and has an internet connection.
Main Flow	1. User types a movie title into the search field. 2. User clicks "Search" or presses Enter. 3. System validates that the input is not empty. 4. System sends a GET request to the movie API. 5. System receives a JSON response and renders matching movie cards.
Alternate Flow	3a. If input is empty, system shows a prompt and does not call the API. 5a. If no matches are found, system displays a "No movies found" message. 5b. If the API call fails, system displays an error message.
Postcondition	Matching movie results (or an appropriate message) are displayed to the user.

8.3 UC-2: View Movie Details

Use Case ID	UC-2
Actor	End User
Precondition	A list of search results is currently displayed.
Main Flow	1. User clicks/taps on a movie card. 2. System sends a detail request to the API using the movie's unique ID. 3. System displays the detail view with poster, rating, genre, runtime, and plot.
Alternate Flow	2a. If the detail request fails, system shows an error and keeps the results list visible.
Postcondition	Full movie details are shown to the user.

8.4 UC-3: Reuse Recent Search

Use Case ID	UC-3
Actor	End User
Precondition	At least one prior search exists in Local Storage.
Main Flow	1. User views the recent searches list. 2. User clicks a past search term. 3. System re-executes the search automatically (as per UC-1, steps 3-5).
Postcondition	Results for the selected past search term are displayed.

9. UI / UX Design Requirements

9.1 Design Theme

The application shall use a clean, modern, card-based layout with a cinematic dark or light theme (developer's choice), consistent spacing, and clear typographic hierarchy. Rounded corners, subtle shadows, and hover animations on movie cards are recommended to enhance perceived quality.

9.2 Key Screens

Screen	Key Elements
Home / Search Screen	App logo/title, search input with placeholder text ("Search for a movie..."), Search button, results grid area.
Results Grid	Responsive grid of movie cards (poster, title, year); adjusts from 1 column on mobile to 4-5 columns on desktop.
Movie Detail View	Modal or dedicated panel showing large poster, title, rating badge, genre tags, runtime, full plot, close/back button.
Empty / Loading / Error States	Centered icon + message for "start searching," animated spinner while loading, and a friendly error illustration/message on failure.

9.3 Responsiveness Requirements

- Mobile (<600px): single-column card layout, collapsible/simplified header.
- Tablet (600-1024px): 2-3 column grid.
- Desktop (>1024px): 4-5 column grid with hover effects enabled.
- All touch targets (buttons, cards) shall be at least 44x44px on mobile for accessibility.

9.4 Accessibility Guidelines

- All images shall include descriptive `alt` attributes (e.g., "Poster of Inception (2010)").
- The search input shall be keyboard-navigable and support the Enter key for submission.
- Color contrast shall meet WCAG AA standards for text on background.

10. Software Testing Plan

10.1 Testing Approach

Testing will primarily be **manual/functional testing**, supplemented with basic unit tests for utility functions (e.g., debounce, input sanitization). This approach is appropriate for the scope and timeline of a college project while still demonstrating sound QA practice.

10.2 Test Case Summary

Test ID	Test Scenario	Expected Result	Related Req.
TC-01	Search with a valid movie title (e.g., "Inception")	Matching results are displayed with poster, title, year.	FR-1.1-FR-1.3, FR-2.1
TC-02	Search with an empty input and click Search	System shows a prompt; no API call is made.	FR-1.4
TC-03	Search with a title that has no matches (e.g., "xyzasdqwe")	"No movies found" message is displayed.	FR-2.4
TC-04	Disconnect internet and perform a search	User-friendly network error message is shown.	FR-4.1
TC-05	Click a movie card from results	Detail view opens with correct, complete movie information.	FR-3.1-FR-3.2
TC-06	Movie with missing poster in API response	Placeholder image is shown instead of a broken image icon.	FR-2.3
TC-07	Resize browser from desktop to mobile width	Layout reflows correctly with no horizontal scroll or overlap.	NFR-8
TC-08	Perform 3 different searches, then reload the page	Recent search history persists via Local Storage.	FR-5.1
TC-09	Rapidly type in the search box (if live search enabled)	API is not called on every keystroke; debounce delay is respected.	FR-1.5
TC-10	Inspect rendered HTML after search with special characters (e.g., <script>)	Input is safely rendered/escaped; no script execution occurs.	NFR-11

10.3 Testing Levels

- **Unit Testing:** Individual JS utility functions (sanitize input, debounce, format rating).
- **Integration Testing:** API service module correctly feeding data into the DOM rendering module.
- **System Testing:** Full end-to-end flow — search, view results, view details, handle errors.
- **Usability Testing:** A few classmates/users test the app and provide feedback on ease of use.
- **Cross-browser/Responsive Testing:** Verify on Chrome, Firefox, and at least one mobile device/emulator.

11. Project Constraints, Assumptions & Future Scope

11.1 Constraints

- Development timeline limited to a single academic semester/term.
- Team size limited to individual or small student groups (typically 1-3 members).
- No budget for paid/premium API tiers — must operate within free API rate limits.
- No dedicated backend server or hosting budget beyond free static-hosting platforms.

11.2 Assumptions

- The chosen movie API's free tier provides sufficient data coverage for demonstration purposes.
- Evaluators will test the application with an active internet connection.
- Browser compatibility is limited to modern, evergreen browsers (no legacy Internet Explorer support required).

11.3 Future Scope (Phase 2 and Beyond)

Enhancement	Description
User Authentication	Allow users to sign up/log in to save personalized favorites and watch-lists.
Backend & Database	Introduce a Node.js/Express backend with MongoDB or Firebase to persist user data securely (API key hidden server-side).
Advanced Filters	Filter search results by genre, release year range, or minimum rating.
Trailer Integration	Embed YouTube trailers within the movie detail view.
Recommendation Engine	Suggest similar movies based on genre or viewing history.
Progressive Web App (PWA)	Enable offline caching and "Add to Home Screen" functionality.

12. Appendix

12.1 Development Milestones (Suggested Timeline)

Phase	Deliverable	Approx. Duration
Phase 1	Requirement gathering & this SRS document	Week 1
Phase 2	UI wireframes + static HTML/CSS layout	Week 2
Phase 3	API integration & dynamic search functionality	Week 3-4
Phase 4	Movie detail view, error handling, responsiveness	Week 5
Phase 5	Testing, bug fixing, and documentation	Week 6
Phase 6	Final presentation & deployment	Week 7

12.2 Approval

This SRS is considered a living document during development and may be revised with faculty guide approval as the project progresses.

Prepared by (Student Developer)

Reviewed & Approved by (Faculty Guide)